Monitor de Ruido en Aulas

Sistema de monitorización acústica en tiempo real

Documentación técnica completa

Centro:	IES Enlaces
Repositorio:	https://github.com/Rubenbros/microfonoAula
Stack:	ESP32 + MQTT + Node.js + Next.js
Fecha:	Marzo 2026

Índice general

1. Introducción **2**
1.1. Descripción del proyecto . . . 2
1.2. Arquitectura general . . . 2
1.3. Stack tecnológico . . . 3

2. Hardware **4**
2.1. M5Stack ATOM Echo S3R (Micrófono distribuido) . . . 4
2.1.1. Configuración GPIO (I2S) . . . 4
2.1.2. Indicador LED . . . 4
2.1.3. Disposición en el aula . . . 5
2.2. M5Stack Core2 v1.1 (Micrófono central) . . . 5

3. Firmware **6**
3.1. Visión general . . . 6
3.2. Filtro A-weighting (dsp.h) . . . 6
3.2.1. Fundamento teórico . . . 6
3.2.2. Secciones del filtro . . . 7
3.2.3. Implementación . . . 7
3.2.4. Coeficientes . . . 7
3.3. Pipeline de medición . . . 8
3.4. Calibración . . . 8
3.4.1. Offset estático (DB_OFFSET) . . . 8
3.4.2. Modo calibración interactivo . . . 8
3.4.3. Calibración por micro en el backend . . . 9
3.5. Conectividad WiFi . . . 9
3.6. Protocolo MQTT . . . 9
3.6.1. Topic . . . 9
3.6.2. Payload JSON . . . 9
3.7. Configuración (config.h) . . . 10
3.8. Compilación (platformio.ini) . . . 10

4. Herramienta de flasheo **11**
4.1. flash.py . . . 11
4.1.1. Características . . . 11
4.1.2. Uso . . . 11
4.1.3. Credenciales WPA2-Enterprise . . . 12

5. Backend **13**
5.1. Arquitectura . . . 13
5.2. Base de datos . . . 13
5.2.1. Esquema . . . 13
5.2.2. Optimizaciones . . . 13
5.3. API REST . . . 14
5.3.1. Ejemplo de respuesta: /api/rooms . . . 14
5.4. Agregación: mediana vs. media . . . 15
5.5. Protocolo WebSocket . . . 15
5.6. Broker MQTT integrado . . . 15
5.7. Detección de micrófonos offline . . . 16
5.8. Variables de entorno . . . 16
5.9. Dependencias . . . 16

6. Frontend **17**
6.1. Arquitectura . . . 17
6.2. Estructura de componentes . . . 17
6.3. Vistas del dashboard . . . 17
6.3.1. Vista general (Dashboard) . . . 17
6.3.2. Vista detallada del aula . . . 18
6.3.3. Vista detallada del micrófono . . . 18
6.4. Hook useNoiseData . . . 18
6.5. Estilos y colores . . . 19
6.6. Dependencias . . . 19

7. Simulador **20**
7.1. Descripción . . . 20
7.2. Escenarios realistas . . . 20
7.3. Uso . . . 20

8. Despliegue **22**
8.1. Requisitos previos . . . 22
8.2. Escenario 1: Producción con hardware real . . . 22
8.2.1. Paso 1: Flashear los micrófonos . . . 22
8.2.2. Paso 2: Arrancar el servidor . . . 22
8.2.3. Paso 3: Acceder al dashboard . . . 22
8.3. Escenario 2: Demo sin hardware . . . 23
8.4. Escenario 3: Docker . . . 23
8.5. Puertos utilizados . . . 23

9. Flujo de datos **24**
9.1. Camino en tiempo real . . . 24
9.2. Camino histórico . . . 24
9.3. Fallback por polling . . . 25

10. Estructura de ficheros **26**

11. Decisiones técnicas **28**
11.1. Mediana vs. media aritmética 28
11.2. A-weighting vs. ponderación plana 28
11.3. Media de 5 segundos 28
11.4. WebSocket + REST fallback 28
11.5. SQLite vs. bases de datos externas 29
11.6. Broker MQTT integrado 29
11.7. Calibración dual 29

A. Referencia rápida de comandos **30**

B. Glosario **31**

Capítulo 1

Introducción

1.1. Descripción del proyecto

El **Monitor de Ruido en Aulas** es un sistema de monitorización acústica en tiempo real diseñado para medir, registrar y visualizar los niveles de ruido en aulas escolares. El sistema utiliza microcontroladores M5Stack (basados en ESP32) equipados con micrófonos MEMS integrados que capturan audio, aplican filtrado A-weighting conforme a la norma IEC 61672, y transmiten las mediciones vía MQTT a un servidor central.

El objetivo principal es proporcionar al profesorado y equipo directivo una herramienta para:

- Monitorizar en tiempo real el nivel de ruido de cada aula.
- Detectar picos de ruido y patrones temporales.
- Comparar niveles entre diferentes zonas del aula (micrófonos distribuidos).
- Almacenar histórico de mediciones para análisis posterior.
- Visualizar los datos en un dashboard web accesible desde cualquier dispositivo.

1.2. Arquitectura general

El sistema sigue una arquitectura IoT clásica de tres capas:

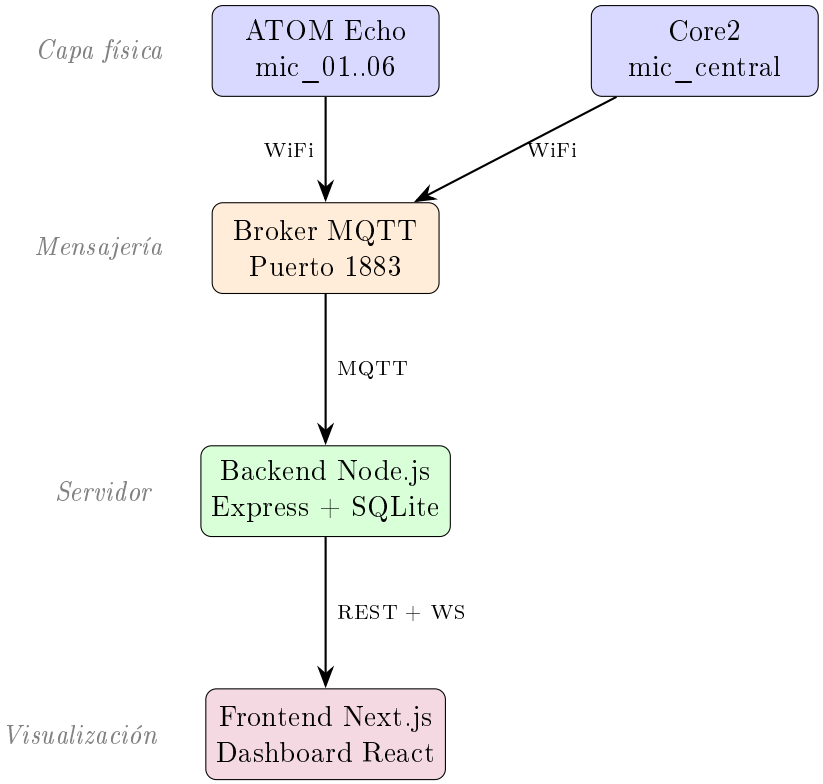

Figura 1.1: Arquitectura general del sistema

1.3. Stack tecnológico

Capa	Tecnología	Versión
Firmware	C++ (Arduino) + PlatformIO	ESP-IDF 5.x
Comunicación	MQTT (Aedes / Mosquitto)	5.x
Backend	Node.js + Express + SQLite	20 LTS
Frontend	Next.js + React + TypeScript	15.1 / 19.0
Visualización	Recharts + Tailwind CSS	2.15 / 3.4
Base de datos	SQLite (better-sqlite3)	11.7

Cuadro 1.1: Stack tecnológico del proyecto

Capítulo 2

Hardware

2.1. M5Stack ATOM Echo S3R (Micrófono distribuido)

El ATOM Echo S3R es el dispositivo principal del sistema. Se despliegan 6 unidades por aula en posiciones estratégicas para cubrir diferentes zonas.

Característica	**Especificación**
MCU	ESP32-S3 (240 MHz dual-core, PSRAM)
Micrófono	PDM MEMS integrado
Interfaz audio	I2S
Conectividad	WiFi 802.11 b/g/n (2.4 GHz)
LED	RGB integrado (indicador de estado)
Botón	Botón A (calibración / reset pico)
Alimentación	USB-C (5V)
Tamaño	$24 \times 24 \times 17$ mm

Cuadro 2.1: Especificaciones del ATOM Echo S3R

2.1.1. Configuración GPIO (I2S)

Señal	**GPIO**	**Función**
LRCK	GPIO 3	Left/Right Clock
SCLK	GPIO 17	Serial Clock
MCLK	GPIO 11	Master Clock
DSDIN	GPIO 48	Data Serial In

Cuadro 2.2: Pines I2S del ATOM Echo S3R

2.1.2. Indicador LED

El LED RGB indica el nivel de ruido actual:

- **Verde**: < 50 dBA (ambiente tranquilo)

- **Amarillo**: 50 – 70 dBA (nivel moderado)
- **Rojo**: > 70 dBA (nivel alto)
- **Azul**: Modo calibración / warmup

2.1.3. Disposición en el aula

Se colocan 6 micrófonos ATOM Echo en una cuadrícula 2×3 para cubrir la superficie del aula:

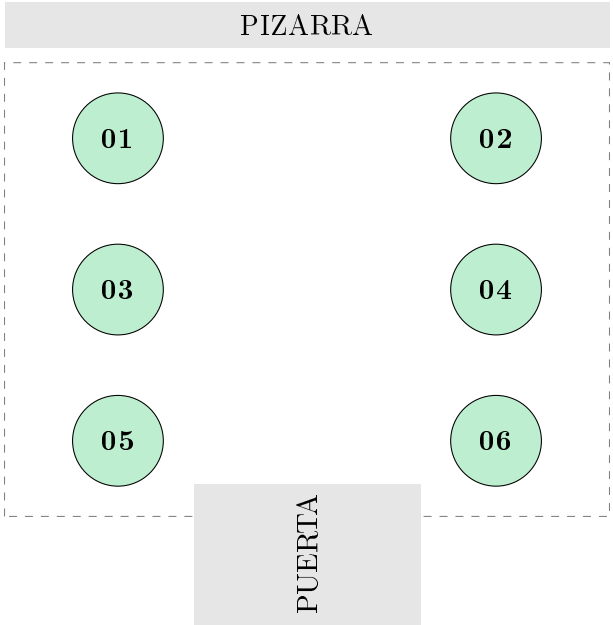

Figura 2.1: Disposición de los 6 micrófonos distribuidos en el aula

2.2. M5Stack Core2 v1.1 (Micrófono central)

El Core2 se utiliza como micrófono central opcional, ofreciendo información visual en su pantalla LCD.

Característica	Especificación
MCU	ESP32 (240 MHz dual-core, 8 MB PSRAM)
Micrófono	SPM1423 (PDM)
Pantalla	LCD IPS 320×240 (16-bit color)
Identificador	`mic_central`
Interfaz	Muestra dBA, RSSI WiFi, uptime, estado

Cuadro 2.3: Especificaciones del M5Stack Core2

Capítulo 3

Firmware

3.1. Visión general

El firmware se ejecuta en los microcontroladores ESP32 y realiza las siguientes funciones:

1. Captura de audio PCM a 16 kHz mediante I2S (M5Unified).
2. Filtrado A-weighting IIR para ponderación frecuencial.
3. Cálculo de nivel RMS y conversión a dBA.
4. Acumulación de mediciones durante 5 segundos.
5. Publicación MQTT con la media y el pico del intervalo.
6. Indicación visual mediante LED RGB.

3.2. Filtro A-weighting (dsp.h)

3.2.1. Fundamento teórico

La ponderación A (A-weighting) es un filtro frecuencial definido en la norma **IEC 61672** que modela la respuesta del oído humano. Atenúa las frecuencias bajas (< 500 Hz) y muy altas (> 6 kHz) donde el oído es menos sensible.

La implementación utiliza una cascada de **3 secciones biquad IIR** en forma *Direct Form II Transposed*:

$$H(z) = H_1(z) \cdot H_2(z) \cdot H_3(z) \cdot G \tag{3.1}$$

Donde $G = 1{,}24358$ es la ganancia de normalización a 0 dB en 1 kHz.

3.2.2. Secciones del filtro

Sección	Tipo	Frecuencia corte	Orden
Biquad 1	Paso alto	20.6 Hz	2º ($Q = 0{,}5$)
Biquad 2	Paso alto	107.7 Hz	1er
Biquad 3	Paso alto	737.9 Hz	1er

Cuadro 3.1: Secciones del filtro A-weighting

Nota: El polo doble a 12194 Hz se omite por estar por encima de la frecuencia de Nyquist (8 kHz a $f_s = 16$ kHz). Error típico: < 2 dB en el rango 100–8000 Hz.

3.2.3. Implementación

Cada sección biquad procesa una muestra según:

$$y[n] = b_0 \cdot x[n] + s_1[n-1] \tag{3.2}$$

$$s_1[n] = b_1 \cdot x[n] - a_1 \cdot y[n] + s_2[n-1] \tag{3.3}$$

$$s_2[n] = b_2 \cdot x[n] - a_2 \cdot y[n] \tag{3.4}$$

```c
static inline float biquad_process(Biquad* f, float x) {
    float y = f->b0 * x + f->s1;
    f->s1 = f->b1 * x - f->a1 * y + f->s2;
    f->s2 = f->b2 * x - f->a2 * y;
    return y;
}
```

Listing 3.1: Procesado biquad (dsp.h)

3.2.4. Coeficientes

```c
// Biquad 1: 2nd-order HP, fc=20.6Hz, Q=0.5
{ b0=0.991965, b1=-1.983931, b2=0.991965,
  a1=-1.983906, a2=0.983946 }

// Biquad 2: 1st-order HP, fc=107.7Hz
{ b0=0.979286, b1=-0.979286, b2=0.0,
  a1=-0.958573, a2=0.0 }

// Biquad 3: 1st-order HP, fc=737.9Hz
{ b0=0.873487, b1=-0.873487, b2=0.0,
  a1=-0.746974, a2=0.0 }
```

Listing 3.2: Coeficientes del filtro A-weighting para $f_s = 16000$ Hz

3.3. Pipeline de medición

El proceso completo desde la captura de audio hasta el envío MQTT es:

1. **Captura**: 1024 muestras PCM 16-bit a 16 kHz ($\approx$ 64 ms por buffer).
2. **Filtrado**: Cada muestra pasa por las 3 secciones biquad + ganancia.
3. **RMS**: Cálculo del valor eficaz del buffer filtrado.

$$\text{RMS} = \sqrt{\frac{1}{N}\sum_{i=0}^{N-1} x_{\text{filtrado}}[i]^2} \tag{3.5}$$

4. **Conversión a dB**:

$$\text{dBA} = 20 \cdot \log_{10}(\text{RMS}) + \text{DB_OFFSET} \tag{3.6}$$

   Con límites $[20, 120]$ dBA.

5. **Acumulación**: Se acumulan ~80 buffers durante 5 segundos.
6. **Envío MQTT**: Media aritmética del intervalo + pico máximo.

3.4. Calibración

3.4.1. Offset estático (DB_OFFSET)

Cada micrófono tiene un parámetro `DB_OFFSET` en `config.h` que permite compensar diferencias de sensibilidad:

```
#define DB_OFFSET 0.0  // Ajustar segun microfono
```

3.4.2. Modo calibración interactivo

Manteniendo pulsado el **Botón A** durante 2 segundos se activa el modo calibración:

1. El dispositivo mide durante 10 segundos.
2. Calcula la media de todas las lecturas.
3. Muestra por puerto serie el resultado y la fórmula de ajuste.

$$\text{DB_OFFSET}_{\text{nuevo}} = \text{DB_OFFSET}_{\text{actual}} + (\text{dB}_{\text{referencia}} - \text{dB}_{\text{medido}}) \tag{3.7}$$

3.4.3. Calibración por micro en el backend

Además del offset en firmware, el backend dispone de una tabla de calibración que aplica correcciones sin necesidad de reflashear:

```
const MIC_CALIBRATION = {
    "mic_central": 7.0,  // Resta 7 dB al central
};

function applyCalibration(mic, dbValue) {
    const offset = MIC_CALIBRATION[mic] || 0;
    return Math.round((dbValue - offset) * 10) / 10;
}
```

Listing 3.3: Calibración en el backend (index.js)

3.5. Conectividad WiFi

El firmware soporta dos modos de autenticación:

Parámetro	WPA2-Personal	WPA2-Enterprise
`WIFI_ENTERPRISE`	`false`	`true`
`WIFI_SSID`	Nombre de la red	Nombre de la red
`WIFI_USER`	(vacío)	Usuario EAP-PEAP
`WIFI_PASSWORD`	Clave WiFi	Clave EAP

Cuadro 3.2: Modos de autenticación WiFi

Estrategia de reconexion: 90 intentos con 1 segundo entre cada uno. Cada 30 intentos realiza un ciclo completo de desconexión y reconexion. Si agota los intentos, reinicia el ESP32.

3.6. Protocolo MQTT

3.6.1. Topic

`aulas/{ROOM_ID}/{MIC_ID}/noise`

Ejemplo: `aulas/aula_01/mic_03/noise`

3.6.2. Payload JSON

```
{
    "room": "aula_01",
    "mic": "mic_03",
    "db": 52.3,
    "peak": 67.1,
    "timestamp": 1773942550
}
```

Listing 3.4: Mensaje MQTT de un micrófono

3.7. Configuración (config.h)

```
// WiFi
#define WIFI_ENTERPRISE false
#define WIFI_SSID "IOT2"
#define WIFI_USER ""
#define WIFI_PASSWORD "IOT_Enlaces_205"

// MQTT
#define MQTT_BROKER "broker.hivemq.com"
#define MQTT_PORT 1883

// Identificacion
#define ROOM_ID "aula_01"
#define MIC_ID "mic_06"

// Medicion
#define SEND_INTERVAL_MS 5000
#define SAMPLE_RATE 16000
#define SAMPLES_PER_READ 1024
#define DB_OFFSET 0.0

// Umbrales
#define THRESHOLD_LOW 50.0
#define THRESHOLD_HIGH 70.0
```

Listing 3.5: Archivo de configuración completo

3.8. Compilación (platformio.ini)

```
[env:atom-echo-s3r]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
monitor_speed = 115200
build_flags =
    -DARDUINO_USB_CDC_ON_BOOT=1
    -DBOARD_HAS_PSRAM
    -DCORE_DEBUG_LEVEL=1
lib_deps =
    m5stack/M5Unified@^0.2.2
    knolleary/PubSubClient@^2.8
    bblanchon/ArduinoJson@^7.2.1
```

Listing 3.6: Configuración PlatformIO para ATOM Echo S3R

Capítulo 4

Herramienta de flasheo

4.1. flash.py

Script Python para flashear múltiples dispositivos ATOM Echo de forma automatizada.

4.1.1. Características

- Autodetección de puertos COM (CP210x, CH340, JTAG, USB genérico).
- Modificación dinámica de `config.h` antes de compilar.
- Soporte para flasheo secuencial de los 6 micrófonos (`-all`).
- Credenciales WPA2-Enterprise prealmacenadas por micro.
- Configuración WiFi por línea de comandos.

4.1.2. Uso

```
# Flashear un micro con credenciales Enterprise prealmacenadas
python flash.py mic_01

# Flashear con WiFi personalizado (WPA2-Personal)
python flash.py mic_01 --wifi MiRed MiClave

# Flashear con WiFi Enterprise (3 parametros)
python flash.py mic_01 --wifi ENLACES-A204 dam251v Shb*XEdv

# Flashear todos los micros
python flash.py --all --wifi MiRed MiClave

# Listar puertos COM disponibles
python flash.py --list
```

Listing 4.1: Ejemplos de uso de flash.py

4.1.3. Credenciales WPA2-Enterprise

MIC_ID	Usuario	Contraseña
mic_01	dam251v	Shb*XEdv
mic_02	dam252v	blG1K1vT
mic_03	dam253v	1B6 %CHbl
mic_04	dam254v	8tE %Phs@
mic_05	dam255v	%@CXFvVm
mic_06	dam256v	JTg71@*b

Cuadro 4.1: Credenciales Enterprise por micrófono

Capítulo 5

Backend

5.1. Arquitectura

El backend es un servidor **Node.js** que integra:

- **Express.js**: API REST (puerto 3001).
- **WebSocket** (ws): Actualizaciones en tiempo real (puerto 3002).
- **Cliente MQTT**: Recibe datos de los micrófonos.
- **SQLite**: Almacenamiento persistente con modo WAL.
- **Broker MQTT integrado** (Aedes, opcional): Elimina la dependencia de Docker/Mosquitto.

5.2. Base de datos

5.2.1. Esquema

```sql
CREATE TABLE noise_readings (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    room        TEXT    NOT NULL,
    mic         TEXT    NOT NULL DEFAULT 'mic_01',
    db_level    REAL    NOT NULL,
    peak_level  REAL    NOT NULL,
    timestamp   INTEGER NOT NULL,
    created_at  TEXT    DEFAULT (datetime('now'))
);

CREATE INDEX idx_room_mic_timestamp
    ON noise_readings (room, mic, timestamp DESC);
```

Listing 5.1: Esquema de la base de datos SQLite

5.2.2. Optimizaciones

- **WAL mode**: Permite lecturas concurrentes durante escrituras.

- **Synchronous NORMAL**: Balance entre rendimiento y seguridad.
- **Prepared statements**: Consultas precompiladas para rendimiento.
- **Índice compuesto**: Búsquedas rápidas por aula, micro y rango temporal.

5.3. API REST

Método	Endpoint	Descripción
GET	`/api/rooms`	Lista de aulas con resumen actual
GET	`/api/rooms/:id`	Detalle de un aula (todos sus micros)
GET	`/api/rooms/:id/history`	Histórico de un aula (paráms: from, to)
GET	`/api/rooms/:id/mics/:micId/history`	Histórico de un micro específico
GET	`/api/stats`	Estadísticas globales (parám: since)
GET	`/api/health`	Estado del sistema (MQTT, uptime)

Cuadro 5.1: Endpoints de la API REST

5.3.1. Ejemplo de respuesta: /api/rooms

```
{
  "rooms": [
    {
      "room": "aula_01",
      "db": 43.9,
      "peak": 73.8,
      "micCount": 7,
      "onlineCount": 7,
      "mics": [
        {
          "room": "aula_01",
          "mic": "mic_central",
          "db": 38.1,
          "peak": 72.3,
          "timestamp": 1773942550,
          "online": true
        }
      ]
    }
  ]
}
```

Listing 5.2: Respuesta del endpoint /api/rooms

5.4. Agregación: mediana vs. media

El valor de dB mostrado para cada aula se calcula como la **mediana** de los micrófonos online, en lugar de la media aritmética. Esto proporciona robustez frente a micrófonos atípicos (outliers):

```
function median(values) {
    if (values.length === 0) return 0;
    const sorted = [...values].sort((a, b) => a - b);
    const mid = Math.floor(sorted.length / 2);
    return sorted.length % 2 !== 0
        ? sorted[mid]
        : (sorted[mid - 1] + sorted[mid]) / 2;
}

const medianDb = onlineMics.length > 0
    ? median(onlineMics.map(m => m.db))
    : 0;
```

Listing 5.3: Cálculo de mediana para agregación del aula

Justificación: Con 7 micrófonos, si uno de ellos (p.ej. mic_central) lee sistemáticamente más alto por estar más cerca de la fuente de sonido, la mediana ignora este valor extremo y refleja mejor el nivel general del aula.

5.5. Protocolo WebSocket

La comunicación WebSocket permite actualizaciones en tiempo real sin polling:

Tipo de mensaje	Descripción
`init`	Estado completo de todas las aulas (al conectar)
`room_update`	Actualización de una aula (cada 5s o al cambiar estado)
`mic_update`	Actualización individual de un micrófono

Cuadro 5.2: Tipos de mensaje WebSocket

5.6. Broker MQTT integrado

Si se establece `USE_INTERNAL_BROKER=true`, el backend arranca un broker MQTT (Aedes) en el puerto 1883, eliminando la necesidad de Docker o Mosquitto externo:

```
const aedes = require("aedes")();
const net = require("net");
const PORT = parseInt(process.env.MQTT_INTERNAL_PORT || "1883");

const server = net.createServer(aedes.handle);
server.listen(PORT, () => {
    console.log(`[BROKER] MQTT broker en puerto ${PORT}`);
});
```

Listing 5.4: Broker MQTT integrado (broker.js)

5.7. Detección de micrófonos offline

Un micrófono se considera **offline** si no envía datos durante 15 segundos. El backend comprueba cada 5 segundos y notifica a los clientes WebSocket del cambio de estado.

5.8. Variables de entorno

```
MQTT_BROKER=mqtt://localhost
MQTT_PORT=1883
HTTP_PORT=3001
WS_PORT=3002
DB_PATH=./data/noise.db
USE_INTERNAL_BROKER=true
```

Listing 5.5: Archivo .env.example

5.9. Dependencias

Paquete	Versión
aedes	0.50.0
better-sqlite3	11.7.0
express	4.21.2
mqtt	5.10.4
ws	8.18.0
dotenv	16.4.7

Cuadro 5.3: Dependencias del backend

Capítulo 6

Frontend

6.1. Arquitectura

El frontend es una aplicación **Next.js 15** con el App Router de React 19. Utiliza:

- **TypeScript**: Tipado estático en todo el código.
- **Tailwind CSS**: Estilos utility-first con colores personalizados.
- **Recharts**: Gráficas interactivas de nivel de ruido.
- **WebSocket**: Actualizaciones en tiempo real.
- **REST fallback**: Polling cada 5s si WebSocket no está disponible.

6.2. Estructura de componentes

Componente	Función
`page.tsx`	Dashboard principal: grid de aulas, barra de estado
`layout.tsx`	Cabecera con logo, leyenda de colores, indicador de conexión
`RoomCard.tsx`	Tarjeta por aula: valor dB grande, pico, sparkline, conteo de micros
`RoomDetailView.tsx`	Vista detallada: mapa 2×3, micro central, estadísticas
`MicDetailView.tsx`	Detalle individual: valor dB, gráfica histórica
`NoiseChart.tsx`	Gráfica Recharts: área dB + línea pico + líneas referencia
`useNoiseData.ts`	Hook personalizado: WebSocket, REST, navegación, estado

Cuadro 6.1: Componentes del frontend

6.3. Vistas del dashboard

6.3.1. Vista general (Dashboard)

Muestra una cuadrícula responsive (1–4 columnas) con una tarjeta por aula. Cada tarjeta incluye:

- Número grande con el nivel actual en dBA (con animación pulse).
- Valor pico.
- Tiempo desde la última lectura ("hace X segundos").
- Conteo de micrófonos online/total.
- Sparkline con las últimas 30 lecturas.
- Color de fondo según nivel (verde/amarillo/rojo).

Barra superior de estado: indicador de conexión, total de aulas, media global, aulas por encima del umbral.

6.3.2. Vista detallada del aula

Al hacer clic en una tarjeta se accede a la vista detallada que muestra:

- **Micrófonos distribuidos**: Mapa 2×3 con la posición real de cada micro. Círculos coloreados con el valor dB. Los micros offline aparecen en gris.
- **Micrófono central** (si existe `mic_central`): Círculo grande con gradiente, comparativa entre media de los distribuidos y el central, diferencia.
- Tarjetas de estadísticas: media, pico máximo, conteo de micros online.
- Navegación breadcrumb.

6.3.3. Vista detallada del micrófono

Al hacer clic en un micrófono individual:

- Valor dB grande con fondo coloreado.
- Gráfica histórica (Recharts) con área para dB, línea para pico, y líneas de referencia a 50 y 70 dB.
- Estadísticas: media, máximo, mínimo del periodo.

6.4. Hook useNoiseData

El hook centraliza toda la gestión de estado y comunicación:

```
interface NoiseDataState {
    rooms: Map<string, RoomSummary>;
    connected: boolean;
    view: "dashboard" | "room" | "mic";
    selectedRoom: string | null;
    selectedMic: string | null;
    history: HistoryReading[];
    loading: boolean;
}
```

Listing 6.1: Interfaz del estado (useNoiseData.ts)

Funciones expuestas:

- `selectRoom(id)`: Navega a la vista del aula.
- `selectMic(room, mic)`: Carga histórico y navega a vista del micro.
- `goBack()`: Navega hacia atrás en la jerarquía.
- `getSparkline(room)`: Devuelve array de últimos 30 valores dB.

6.5. Estilos y colores

Nivel	Rango	Color	Código
Bajo	< 50 dBA	Verde	`#22c55e`
Medio	$50 - 70$ dBA	Amarillo	`#eab308`
Alto	> 70 dBA	Rojo	`#ef4444`

Cuadro 6.2: Escala de colores según nivel de ruido

6.6. Dependencias

Paquete	Versión
next	15.1.0
react	19.0.0
recharts	2.15.0
tailwindcss	3.4.16
typescript	5.7.2

Cuadro 6.3: Dependencias del frontend

Capítulo 7

Simulador

7.1. Descripción

El simulador genera datos de ruido realistas sin necesidad de hardware físico. Útil para:

- Demostraciones y presentaciones.
- Desarrollo y pruebas del frontend/backend.
- Validación de la cadena completa de datos.

7.2. Escenarios realistas

Escenario	**dB base**	**± dB**	**Descripción**
Clase tranquila	35	8	Explicación en voz baja
Explicación profesor	50	10	Clase magistral normal
Trabajo en grupo	58	12	Actividad colaborativa
Debate activo	65	10	Participación alta
Recreo	72	15	Hora de descanso
Examen	28	5	Silencio máximo
Aula vacía	20	3	Sin ocupantes

Cuadro 7.1: Escenarios de simulación

7.3. Uso

```
# Por defecto: 6 aulas, intervalo 2s
node simulate.js

# 12 aulas con intervalo de 1 segundo
node simulate.js --rooms 12 --interval 1000

# Broker personalizado
node simulate.js --broker mqtt://192.168.1.50
```

Listing 7.1: Ejecución del simulador

El simulador incluye 20 nombres de aulas predefinidos (aula_101 a aula_208, laboratorios, biblioteca, gimnasio, etc.) y genera transiciones suaves entre escenarios con picos aleatorios y decaimiento exponencial.

Capítulo 8

Despliegue

8.1. Requisitos previos

- **Node.js** $\geq$ 18 (recomendado 20 LTS o 22).
- **Python** $\geq$ 3.8 con `platformio` y `pyserial` (solo para flasheo).
- **Docker** (opcional, solo si se usa Mosquitto externo).

8.2. Escenario 1: Producción con hardware real

8.2.1. Paso 1: Flashear los micrófonos

```
cd firmware
pip install platformio pyserial
python flash.py mic_01  # Repetir para mic_02..06
```

8.2.2. Paso 2: Arrancar el servidor

Windows:

```
start.bat
```

Linux/Mac:

```
chmod +x start.sh && ./start.sh
```

Esto arranca:

1. Backend con broker MQTT integrado (puertos 1883, 3001, 3002).
2. Frontend en modo desarrollo (puerto 3000).

8.2.3. Paso 3: Acceder al dashboard

- Dashboard: http://localhost:3000
- API: http://localhost:3001/api/rooms
- MQTT: `localhost:1883`

8.3. Escenario 2: Demo sin hardware

```
# Windows
scripts\demo.bat

# Linux/Mac
./scripts/demo.sh
```

Arranca el simulador en lugar de esperar datos de micrófonos reales.

8.4. Escenario 3: Docker

```
docker compose up -d    # Mosquitto + Backend
cd frontend && npm run dev  # Frontend aparte
```

8.5. Puertos utilizados

Puerto	Servicio	Protocolo
1883	Broker MQTT	TCP
3000	Frontend (Next.js)	HTTP
3001	Backend API (Express)	HTTP
3002	WebSocket	WS
9001	MQTT WebSocket (Mosquitto)	WS

Cuadro 8.1: Puertos del sistema

Capítulo 9

Flujo de datos

9.1. Camino en tiempo real

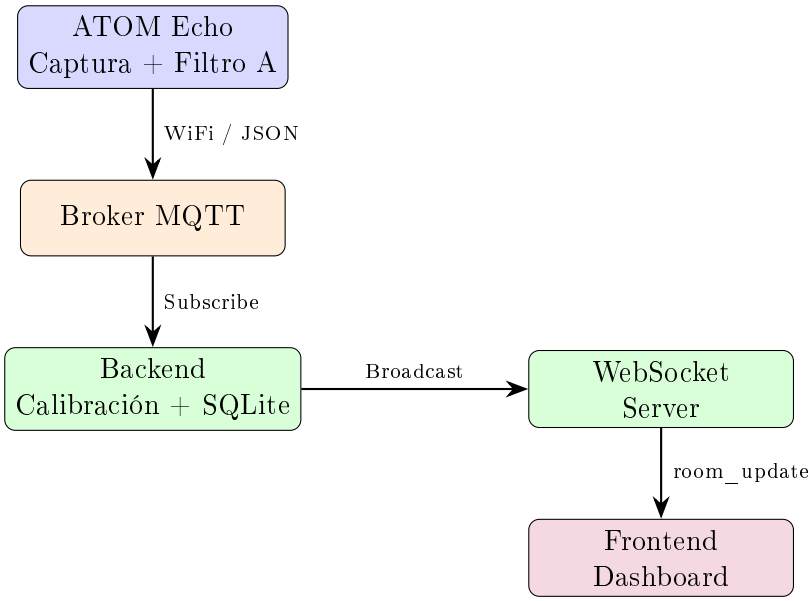

Figura 9.1: Flujo de datos en tiempo real

9.2. Camino histórico

Cuando el usuario hace clic en un micrófono para ver su histórico:

1. El frontend llama a `GET /api/rooms/:id/mics/:micId/history`.
2. El backend consulta SQLite con el rango temporal.
3. Devuelve array de lecturas con timestamps.
4. El componente `NoiseChart` renderiza la gráfica.

9.3. Fallback por polling

Si la conexión WebSocket se pierde, el hook `useNoiseData` activa un polling REST cada 5 segundos a `/api/rooms` hasta que el WebSocket se reconecta (reintento automático cada 3 segundos).

Capítulo 10

Estructura de ficheros

```
microfonoAula/
|-- firmware/                       # ATOM Echo S3R
|   |-- src/
|   |   |-- config.h              # Configuracion WiFi/MQTT/IDs
|   |   |-- dsp.h                 # Filtro A-weighting IIR
|   |   +-- main.cpp              # Logica principal
|   |-- platformio.ini            # Build config
|   +-- flash.py                  # Herramienta de flasheo
|
|-- firmware_core2/                 # M5Stack Core2
|   |-- src/
|   |   |-- config.h
|   |   |-- dsp.h
|   |   +-- main.cpp              # Con LCD display
|   |-- platformio.ini
|   +-- flash_core2.py
|
|-- backend/                        # Servidor Node.js
|   |-- src/
|   |   |-- index.js              # API + WS + MQTT + SQLite
|   |   +-- broker.js             # Broker MQTT integrado
|   |-- package.json
|   |-- .env.example
|   +-- Dockerfile
|
|-- frontend/                       # Dashboard Next.js
|   |-- src/
|   |   |-- app/
|   |   |   |-- page.tsx          # Dashboard principal
|   |   |   |-- layout.tsx        # Cabecera
|   |   |   +-- globals.css       # Estilos globales
|   |   |-- components/
|   |   |   |-- RoomCard.tsx
|   |   |   |-- RoomDetailView.tsx
|   |   |   |-- MicDetailView.tsx
|   |   |   +-- NoiseChart.tsx
|   |   +-- lib/
|   |       +-- useNoiseData.ts   # Hook principal
|   |-- package.json
|   |-- next.config.ts
|   +-- tailwind.config.ts
|
|-- simulator/                      # Simulador sin hardware
|   |-- simulate.js
|   +-- package.json
|
|-- scripts/                        # Scripts de automatizacion
|   |-- demo.bat / demo.sh
|   |-- run_backend.bat
|   +-- run_frontend.bat
|
|-- start.bat / start.sh          # Arranque produccion
|-- setup.bat                       # Instalacion
|-- docker-compose.yml            # Docker (Mosquitto)
```

```
|-- mosquitto.conf                # Config Mosquitto
|-- requirements.txt              # Python deps
|-- README.md                      # Guia rapida
+-- DESPLIEGUE.md                 # Guia de despliegue
```

Listing 10.1: Árbol completo del proyecto

Capítulo 11

Decisiones técnicas

11.1. Mediana vs. media aritmética

Se utiliza la **mediana** para agregar los valores de los micrófonos de un aula en lugar de la media aritmética. Razón: el micrófono central, por su posición más cercana al profesor, registra sistemáticamente valores superiores (4–10 dB según el nivel de actividad). La mediana es robusta frente a estos outliers posicionales.

11.2. A-weighting vs. ponderación plana

Se implementa ponderación A (dBA) porque:

- Es el estándar internacional para medición de ruido ambiental (IEC 61672).
- Refleja la percepción real del oído humano.
- Atenúa el ruido de baja frecuencia (ventilación, tráfico) que no molesta tanto.

11.3. Media de 5 segundos

En lugar de enviar lecturas instantáneas, se acumula durante 5 segundos:

- Reduce el tráfico MQTT ($\sim$12 mensajes/minuto por micro vs. $\sim$900 instantáneos).
- Suaviza fluctuaciones rápidas que no son representativas.
- El pico se preserva por separado para no perder eventos puntuales.

11.4. WebSocket + REST fallback

Se usa WebSocket como canal primario por su baja latencia, con fallback automático a polling REST. Esto garantiza que el dashboard funcione incluso en redes que bloquean WebSocket.

11.5. SQLite vs. bases de datos externas

Se eligió SQLite por:

- Cero configuración (fichero local).
- Rendimiento excelente para un solo servidor.
- Modo WAL para lecturas concurrentes.
- Portabilidad (la base de datos es un único fichero `.db`).

11.6. Broker MQTT integrado

Se incluye Aedes como broker opcional para eliminar la dependencia de Docker/Mosquitto en despliegues simples. El backend puede actuar como broker y cliente MQTT simultáneamente.

11.7. Calibración dual

Se implementan dos niveles de calibración:

1. **Firmware** (`DB_OFFSET`): Compensación estática por hardware. Requiere reflashear.
2. **Backend** (`MIC_CALIBRATION`): Compensación dinámica por posición. Ajuste inmediato sin tocar el hardware.

Apéndice A

Referencia rápida de comandos

Comando	Descripción
`start.bat`	Arrancar backend + frontend (Windows)
`./start.sh`	Arrancar backend + frontend (Linux/Mac)
`scripts/demo.bat`	Demo con simulador (Windows)
`python flash.py mic_01`	Flashear un micrófono
`python flash.py -all`	Flashear todos los micros
`python flash.py -list`	Listar puertos COM
`node simulate.js`	Iniciar simulador (6 aulas)
`docker compose up -d`	Arrancar con Docker
`npm run dev`	Frontend en modo desarrollo
`npm run build`	Compilar frontend para producción

Cuadro A.1: Referencia rápida de comandos

Apéndice B

Glosario

dBA
Decibelios con ponderación A. Unidad de medida de nivel sonoro que tiene en cuenta la sensibilidad del oído humano a distintas frecuencias.

A-weighting
Curva de ponderación frecuencial definida en IEC 61672 que modela la respuesta del oído humano.

Biquad
Filtro digital IIR de segundo orden. Bloque básico para construir filtros más complejos.

RMS
Root Mean Square (valor eficaz). Medida de la amplitud de una señal.

MQTT
Message Queuing Telemetry Transport. Protocolo de mensajería ligero para IoT.

I2S Inter-IC Sound. Protocolo de comunicación serie para audio digital.

PDM
Pulse Density Modulation. Técnica de modulación usada por micrófonos MEMS.

WAL
Write-Ahead Logging. Modo de SQLite que permite lecturas concurrentes durante escrituras.

SPL
Sound Pressure Level. Nivel de presión sonora medido en decibelios.

MEMS
Micro-Electro-Mechanical Systems. Tecnología de fabricación de micrófonos miniaturizados.

EAP-PEAP
Protected Extensible Authentication Protocol. Método de autenticación para WPA2-Enterprise.